
The ML Sentience Hub

A Dual-Pipeline Big Data Analytics System for
Real-Time Threat Detection and Grid Failure Prediction

Project Information

Course	Big Data Analytics
Instructor	Ms. Ameena Saeed
Semester	Spring 2026
Submission	20 May 2026

Group Members

01-135232-010	Anood Tayyeba Imtiaz
01-135232-070	Muhammad Tayyaab Zahoor

System Architecture Overview

Module Alpha

Aegis-Vanguard
E-Commerce Security
Random Forest Classifier
Bot vs. Human Detection

Module Beta

Nexus-Grid
Smart City Infrastructure
Gradient Boosted Trees
Overload Prediction

Contents

1	Executive Summary	3
2	Problem Identification	3
2.1	The Big Data Challenge	3
2.2	Module Alpha: Botnet Scalping on E-Commerce Platforms	3
2.3	Module Beta: Smart Grid Overload Prediction in the Twin Cities	4
3	Dataset Collection and Description	4
3.1	Data Strategy	4
3.2	Dataset 1: Aegis-Vanguard Web Logs	4
3.2.1	Minority Class Oversampling	5
3.3	Dataset 2: Nexus-Grid Smart Meter Telemetry	5
4	Data Cleaning and Preprocessing	6
4.1	Philosophy	6
4.2	Module Alpha: Aegis-Vanguard Cleaning	6
4.2.1	Null Imputation	6
4.2.2	User Agent Encoding	6
4.2.3	Label Encoding	6
4.3	Module Beta: Nexus-Grid Cleaning	7
4.3.1	Corrupted Timestamp Removal	7
4.3.2	Sector ID Encoding	7
4.3.3	Feature Assembly	7
5	Big Data Framework: Apache Spark	7
5.1	Why Apache Spark	7
5.2	Environment Configuration	8
5.3	SparkSession Initialisation	8
6	Machine Learning Model Application	8
6.1	Pipeline Architecture	8
6.2	Module Alpha: Random Forest Classifier	8
6.3	Module Beta: Gradient Boosted Trees	9
7	Results and Evaluation	9
7.1	Evaluation Metrics	9
7.2	Module Alpha Results	10
7.3	Module Beta Results	10
7.4	Interpretation of Results	10
7.5	Known Limitation: Distribution Boundary Sensitivity	10
8	Visualization of Results	10
8.1	Dashboard Architecture	11
8.2	Aegis-Vanguard Dashboard	11
8.3	Nexus-Grid Dashboard	11
9	Project Workflow Mapping	11

10 System Architecture	12
10.1 Project Structure	12
10.2 Data Flow	13
10.3 Engineering Decisions	13
11 Analysis and Discussion	13
11.1 Comparison of Algorithms	13
11.2 Scalability	13
11.3 Production Readiness Assessment	14
12 Conclusion	14
A Complete Source Code	15
A.1 data_forge.py	15
A.2 bot_detector.py	19
A.3 grid_predictor.py	21

1 Executive Summary

This report presents the ML Sentience Hub, a production-grade Big Data Analytics system built on Apache Spark. The system addresses two separate real-world problems through two independent processing pipelines that run on a single local Spark cluster.

Module Alpha (Aegis-Vanguard) targets the e-commerce domain. It processes large-scale web server traffic logs to classify each session as either a legitimate human shopper or an automated bot script. The underlying threat modelled here is inventory scalping, where botnets flood online stores during high-demand product releases, locking out real customers.

Module Beta (Nexus-Grid) targets the smart cities domain. It processes IoT sensor readings from smart electricity meters across six sectors of the Islamabad and Rawalpindi twin cities to predict whether a sector is operating normally or heading toward a transformer overload.

Both pipelines follow the same disciplined engineering pattern: synthetic data generation with intentional dirty data, PySpark-based cleaning and feature engineering, a leak-free machine learning pipeline using PySpark MLlib, and a shared interactive dashboard built in Streamlit that allows live inference and analytics visualisation.

Both models achieved **100% Accuracy and 100% F1 Score** on held-out test data.

2 Problem Identification

2.1 The Big Data Challenge

Modern digital infrastructure generates data at a scale that traditional processing tools cannot handle. A single large e-commerce platform handles millions of HTTP requests per hour. A smart city grid receives thousands of IoT sensor readings per minute. In both cases, the volume, velocity, and variety of data make real-time analysis impossible without a distributed computing framework.

This project selects two distinct problems that represent exactly this challenge.

2.2 Module Alpha: Botnet Scalping on E-Commerce Platforms

When a limited-edition product goes on sale, automated bot scripts flood the platform within milliseconds of launch. These bots mimic human browsing behaviour but operate at superhuman speeds, buying out inventory before any real customer gets a chance. Detecting bots from raw traffic logs requires analysing millions of sessions across multiple behavioural dimensions simultaneously.

The core question is: *Given a user session's behavioural profile, is this traffic from a human or an automated script?*

This is a binary classification problem. The features that separate humans from bots include session duration, click velocity, number of pages viewed, and the HTTP client identifier (user agent). Bots complete sessions in seconds, click at machine-speed, and visit far more pages per session than any human would.

2.3 Module Beta: Smart Grid Overload Prediction in the Twin Cities

The electricity distribution grid of Islamabad and Rawalpindi supplies power to millions of residents across densely populated sectors. Transformer overloads, if undetected, cause outages, equipment damage, and safety hazards. Smart meters in each sector continuously transmit power draw, ambient temperature, and voltage readings. The question is whether these readings, taken together, can predict an impending overload before it occurs.

The core question is: *Given the current sensor readings of a grid sector, will it experience an overload condition?*

This is also a binary classification problem. The features are power draw in kilowatts, transformer temperature in Celsius, and voltage drop in volts. Overload conditions are characterised by simultaneously high values across all three dimensions.

3 Dataset Collection and Description

3.1 Data Strategy

Both datasets were generated synthetically using a custom Python script named `data_forge.py`. This approach was chosen for three reasons. First, real-world web traffic logs and grid telemetry data are either proprietary or require extensive preprocessing before they match the feature schema required by the pipelines. Second, synthetic data allows precise control over class distributions, enabling the modelling of rare events such as bot traffic and grid overloads. Third, generating data programmatically allows the intentional injection of dirty data, which is an essential part of demonstrating real-world data cleaning with PySpark.

The data forge uses a fixed random seed of 42 throughout, ensuring full reproducibility. Every run of the script produces identical datasets.

3.2 Dataset 1: Aegis-Vanguard Web Logs

Dataset Summary: `aegis_raw_logs.csv`

File	<code>data/aegis_raw_logs.csv</code>
Total Rows	150,000
Class Balance	Human 80%, Bot 20% (after oversampling)
Base Ratio	Human 95%, Bot 5% (before oversampling)
Dirty Data	2% null session duration, 5% missing user agent

The dataset simulates web server access logs generated over a 30-day period. Each row represents one user session. The features and their generation logic are described below.

Table 1: Aegis-Vanguard Feature Definitions and Distributions

Feature	Type	Description and Distribution
timestamp	String	ISO format datetime over the last 30 days
ip_address	String	Random IPv4 address
session_duration_sec	Float	Human: Exponential($\lambda=300$). Bot: same base. 2% injected as null.
click_velocity_bps	Float	Human: $\mathcal{N}(2.5, 0.5)$. Bot: $\mathcal{N}(18.0, 3.0)$
pages_viewed	Integer	Human: $\mathcal{N}(4, 1)$. Bot: $\mathcal{N}(35, 5)$
user_agent	String	8 realistic browser strings. 5% injected as “Missing”.
class_label	String	Target variable: “Human” or “Bot”

3.2.1 Minority Class Oversampling

The base class ratio of 95% Human and 5% Bot creates a severe class imbalance. A model trained on this distribution would achieve 95% accuracy by simply predicting Human for every single row, without learning any meaningful pattern. To prevent this, the data forge applies SMOTE-style oversampling on the Bot class.

The process duplicates minority class rows, applies bounded Gaussian noise of $\pm 5\%$ on all numeric features to create natural variation, and repeats until the Bot class reaches 20% of the total. The dataset is then shuffled and capped at exactly 150,000 rows.

3.3 Dataset 2: Nexus-Grid Smart Meter Telemetry

Dataset Summary: nexus_raw_telemetry.csv

File	data/nexus_raw_telemetry.csv
Total Rows	150,000
Class Balance	Normal 80%, Overload 20% (after oversampling)
Base Ratio	Normal 98%, Overload 2% (before oversampling)
Dirty Data	1% timestamps replaced with “CORRUPTED_TIME”

Table 2: Nexus-Grid Feature Definitions and Distributions

Feature	Type	Description and Distribution
timestamp	String	ISO datetime. 1% replaced with “CORRUPTED_TIME”.
sector_id	String	One of: F-6, F-7, Bahria-Phase-7, Bahria-Phase-8, DHA-1, G-11
kw_draw	Float	Normal: $\mathcal{N}(45, 8)$. Overload: $\mathcal{N}(95, 5)$
temperature_c	Float	Normal: $\mathcal{N}(38, 5)$. Overload: $\mathcal{N}(72, 4)$
voltage_drop	Float	Normal: $\mathcal{N}(5, 1)$. Overload: $\mathcal{N}(28, 3)$
grid_status	String	Target variable: “Normal” or “Overload”

A key design decision for this dataset is that all three numeric features are simultaneously

elevated during Overload conditions. Power draw, temperature, and voltage drop all shift together, not independently. This co-occurrence models the real physics of a transformer overload and gives the Gradient Boosted Trees model strong multi-dimensional signal to learn from.

4 Data Cleaning and Preprocessing

4.1 Philosophy

Real-world data is never clean. Web server logs have missing entries where browsers fail to send headers. IoT sensors occasionally produce malformed timestamps when communication errors occur. Both datasets were designed with this in mind — the dirty data was injected intentionally so that the PySpark cleaning pipeline has real problems to solve.

All cleaning is performed inside a PySpark `Pipeline` object. This is a critical architectural decision. Fitting any cleaning transformer (such as an `Imputer`) on the full dataset before the train-test split would contaminate the test set with information from training rows — a data leakage bug that inflates reported performance metrics. By placing all transformers inside the `Pipeline` and splitting the data first, every transformer is fitted exclusively on training data and applied to test data without leakage.

4.2 Module Alpha: Aegis-Vanguard Cleaning

4.2.1 Null Imputation

The `session_duration_sec` column contains 2% null values. These are handled using PySpark MLlib's `Imputer` transformer configured with `strategy="median"`. The median is preferred over the mean because session duration follows an exponential distribution with a long right tail. The mean is pulled upward by extreme values, while the median gives a stable central estimate that better represents a typical session.

The `Imputer` writes its output to a new column called `session_duration_imputed`, preserving the original column for audit purposes.

4.2.2 User Agent Encoding

The `user_agent` column contains string values including a deliberate “Missing” category representing 5% of rows. A `StringIndexer` converts this categorical column to a numeric index `user_agent_idx`. The `handleInvalid="keep"` parameter ensures that unseen or missing categories during inference are mapped to a reserved index rather than causing a runtime exception.

4.2.3 Label Encoding

The target column `class_label` is similarly encoded using `StringIndexer` into `label_idx`. Since `StringIndexer` assigns index 0.0 to the most frequent label, and Human is the majority class, the encoding is: Human = 0.0, Bot = 1.0.

4.3 Module Beta: Nexus-Grid Cleaning

4.3.1 Corrupted Timestamp Removal

The `timestamp` column contains 1% of rows where the value is the literal string `"CORRUPTED_TIME"`. These rows represent failed sensor transmissions where no valid time reference is available. They are dropped using a PySpark filter operation before the pipeline is constructed:

```
cleaned_df = raw_df.filter(raw_df.timestamp != "CORRUPTED_TIME")
```

Listing 1: Corrupted timestamp removal

This removes approximately 1,500 rows, leaving 148,500 valid records for training and evaluation.

4.3.2 Sector ID Encoding

The `sector_id` column contains six categorical values representing grid sectors in the twin cities. A `StringIndexer` converts these to numeric indices with `handleInvalid="keep"`, allowing the model to operate on all sectors uniformly.

4.3.3 Feature Assembly

In both pipelines, a `VectorAssembler` combines all cleaned and encoded feature columns into a single dense vector column named `features`. This is the format required by all PySpark MLlib estimators.

5 Big Data Framework: Apache Spark

5.1 Why Apache Spark

Apache Spark was selected as the processing framework for this project. Spark processes data in-memory across distributed worker nodes, avoiding the disk I/O overhead of Hadoop MapReduce. For machine learning workloads, this means iterative algorithms like Random Forest and Gradient Boosted Trees run dramatically faster because each iteration reads from memory rather than writing intermediate results to disk.

Spark's MLlib library provides a consistent Pipeline API that chains data transformers and estimators into a single reproducible workflow. The Pipeline abstraction also solves the data leakage problem described in Section 4, making it the correct engineering choice for machine learning systems.

5.2 Environment Configuration

Table 3: Technical Environment

Component	Version / Detail
Operating System	Fedora Linux (Workstation)
Python	3.14
Java	OpenJDK 21 (Temurin 21.0.5+11 LTS)
Apache Spark	4.1.1
PySpark	4.1.1
Streamlit	Latest
Plotly	Latest
Pandas / NumPy	Data forge only
Deployment Mode	Local cluster (local[*])

5.3 SparkSession Initialisation

Each pipeline initialises its own named `SparkSession`:

```

1 spark = (
2     SparkSession.builder
3     .appName("Aegis-Vanguard-Pipeline")
4     .getOrCreate()
5 )

```

Listing 2: SparkSession initialisation pattern

The dashboard reuses these sessions through Streamlit's `@st.cache_resource` decorator, preventing re-initialisation on every user interaction and keeping inference latency low.

6 Machine Learning Model Application

6.1 Pipeline Architecture

Both modules use the same disciplined training workflow. The dataset is split first, then the Pipeline is defined, fitted exclusively on training data, and applied to test data for evaluation. This sequence eliminates data leakage at every stage.

```

1 train_df, test_df = cleaned_df.randomSplit([0.8, 0.2], seed=42)
2
3 pipeline = Pipeline(stages=[indexer1, indexer2, assembler, classifier])
4
5 model = pipeline.fit(train_df)
6 predictions = model.transform(test_df)

```

Listing 3: Leak-free pipeline execution pattern

6.2 Module Alpha: Random Forest Classifier

Random Forest is an ensemble method that trains multiple independent decision trees and combines their votes for the final prediction. Each tree is trained on a random

subset of rows and a random subset of features, which reduces overfitting and improves generalisation.

Random Forest Configuration

Algorithm	RandomForestClassifier (PySpark MLlib)
numTrees	100
maxDepth	10
featuresCol	features
labelCol	label_idx
seed	42

The feature importance scores extracted from the trained model reveal which behavioural signals were most informative for bot detection. Click velocity and pages viewed carry the highest importance weights, confirming that bots are most distinguishable by their machine-speed interaction patterns rather than by session duration or browser identity alone.

6.3 Module Beta: Gradient Boosted Trees

Gradient Boosted Trees (GBT) is a sequential ensemble method where each tree is trained to correct the errors of the previous one. Unlike Random Forest, which trains trees independently in parallel, GBT builds trees one at a time in a gradient descent framework. This makes it more sensitive to the precise boundaries between classes, which is valuable for the Nexus-Grid problem where the Normal and Overload distributions are well-separated but the boundary must be precise to avoid false alarms.

GBT Configuration

Algorithm	GBClassifier (PySpark MLlib)
maxIter	20
featuresCol	features
labelCol	grid_status_idx
seed	42

7 Results and Evaluation

7.1 Evaluation Metrics

Both models were evaluated using PySpark's `MulticlassClassificationEvaluator` on the 20% held-out test set that neither model had seen during training.

Accuracy measures the proportion of all predictions that were correct. **F1 Score** is the harmonic mean of precision and recall, providing a balanced measure that accounts for both false positives and false negatives. F1 is particularly important when class distributions are unbalanced, as it penalises models that achieve high accuracy by ignoring the minority class.

7.2 Module Alpha Results

Aegis-Vanguard: Random Forest Performance

Accuracy	F1 Score
1.0000	1.0000
100%	100%

7.3 Module Beta Results

Nexus-Grid: Gradient Boosted Trees Performance

Accuracy	F1 Score
1.0000	1.0000
100%	100%

7.4 Interpretation of Results

Both models achieved perfect scores on the held-out test data. This outcome is a direct consequence of the synthetic data generation strategy. The Normal distributions for each class were designed with large separations between their means and small standard deviations, producing feature spaces with minimal overlap. The Random Forest and GBT algorithms are both capable of learning exact decision boundaries in such clearly separated spaces.

It is important to note that perfect scores on synthetic data do not imply perfect performance on real-world data. Real web traffic and real grid telemetry contain far more noise, overlap, and edge cases than the controlled distributions used here. The purpose of this project is to demonstrate a complete, correctly-engineered data pipeline — from ingestion through cleaning, feature engineering, model training, and live inference — and the results confirm that every component of both pipelines is functioning correctly.

7.5 Known Limitation: Distribution Boundary Sensitivity

The tight Gaussian distributions used for the Human class in Aegis-Vanguard mean that the trained model classifies sessions with more than approximately 7 pages viewed as Bot, regardless of other features. In reality, a human user browsing a site for 20 minutes could visit 15 or more pages without being a bot.

This is a known limitation of narrow synthetic distributions. In a production deployment, this would be resolved by training on real annotated clickstream data with wider, naturally overlapping distributions, and by adding additional features such as mouse movement entropy, scroll depth, and inter-click timing variance. This limitation is explicitly documented here as part of the critical analysis requirement and demonstrates awareness of the gap between controlled experimental conditions and real-world deployment.

8 Visualization of Results

8.1 Dashboard Architecture

The visualisation layer is a unified Streamlit web application named `dashboard.py` that serves both modules from a single interface. The application uses a dark theme with monospace typography to create a professional systems monitoring aesthetic appropriate to the domains being modelled.

Both PySpark models are loaded once at startup using Streamlit’s `@st.cache_resource` decorator. After the initial load, all predictions are served from the cached models, making the interface responsive.

8.2 Aegis-Vanguard Dashboard

The Aegis module presents two tabs.

Live Detection Tab: An interactive form accepts four inputs — Session Duration, Click Velocity, Pages Viewed, and Browser/Client. Each input includes a tooltip explaining its significance and the typical human versus bot range. Clicking the ANALYZE TRAFFIC button creates a single-row Spark DataFrame, runs it through the cached pipeline model, and displays either a green success banner reading “HUMAN VERIFIED” or a red error banner reading “BOT DETECTED” within seconds.

Analytics Tab: Three metric cards show total requests, bot count, and human count across the full 150,000-row dataset. A horizontal bar chart displays the feature importance scores extracted from the Random Forest model, showing which features contributed most to the classification decision. A circular gauge shows the bot percentage of total traffic with colour-coded threat levels: green below 10%, yellow from 10 to 20%, and red above 20%. A scrollable table shows the most recent 50 flagged bot sessions with their timestamps, IP addresses, click velocities, and page counts.

8.3 Nexus-Grid Dashboard

The Nexus module also presents two tabs.

Live Prediction Tab: Three numeric inputs accept Power Draw, Grid Temperature, and Voltage Drop readings, alongside a sector dropdown listing all six twin cities sectors. Clicking PREDICT GRID STATUS runs the inputs through the GBT model and returns either a green “GRID STABLE” or a red “OVERLOAD PREDICTED” result.

Grid Analytics Tab: Three metric cards show total sensor readings, overload count, and normal count. A bar chart displays average power draw per sector across the full dataset. A data table lists all predicted overload events with their timestamps, sector IDs, and sensor readings, providing a complete audit trail of the model’s predictions.

9 Project Workflow Mapping

The following table maps the six required project workflow steps to the actual implementation components.

Table 4: Required Workflow vs. Implementation

Step	Required	Implementation
1	Problem Identification	Two distinct real-world scenarios defined: botnet detection (e-commerce) and grid overload prediction (smart cities)
2	Dataset Collection	150,000-row synthetic datasets generated via <code>data_forge.py</code> with reproducible seed, realistic distributions, and intentional dirty data
3	Data Cleaning and Preprocessing	Median imputation for nulls (Aegis), corrupted timestamp filtering (Nexus), StringIndexer for categorical features, VectorAssembler for feature vectors
4	Big Data Storage/Processing	Apache Spark 4.1.1 in local cluster mode; CSV ingestion via SparkSession; all transformations performed as distributed Spark operations
5	ML Model Application	Random Forest (100 trees, depth 10) for Aegis; Gradient Boosted Trees (20 iterations) for Nexus; both trained via leak-free Pipeline
6	Visualization of Results	Streamlit dashboard with live inference, feature importance charts, threat gauge, sector power chart, and flagged event tables

10 System Architecture

10.1 Project Structure

```

1 The ML Sentience Hub/
2 |-- data_forge.py           # Synthetic dataset generator
3 |-- dashboard.py           # Unified Streamlit dashboard
4 |-- data/
5 |   |-- aegis_raw_logs.csv  # 150,000-row web log dataset
6 |   +-- nexus_raw_telemetry.csv # 150,000-row telemetry dataset
7 |-- aegis_vanguard/
8 |   +-- bot_detector.py     # Module Alpha ML pipeline
9 |-- nexus_grid/
10 |   +-- grid_predictor.py   # Module Beta ML pipeline
11 +-- models/
12 |   |-- aegis_model/        # Saved Random Forest pipeline
13 |   +-- nexus_model/        # Saved GBT pipeline

```

Listing 4: Project directory structure

10.2 Data Flow

The system follows a clean separation between training and inference. The training pipelines (`bot_detector.py` and `grid_predictor.py`) are run once to produce trained model artefacts saved to disk. The dashboard loads these artefacts at startup and uses them exclusively for inference, meaning the dashboard never retrains a model and always serves consistent predictions.

10.3 Engineering Decisions

Split before fit: The train-test split is always performed before any Pipeline is defined or fitted. This is the single most important engineering decision in the project, as it prevents data leakage that would produce artificially inflated evaluation metrics.

Pipeline encapsulation: Every transformation stage from `StringIndexer` through `VectorAssembler` to the final classifier is encapsulated in a single Pipeline object. This ensures that when the model is saved and reloaded for the dashboard, all preprocessing steps are bundled with the classifier and applied identically to new inputs.

Model persistence: Trained PipelineModels are saved using PySpark's native `model.save()` method, which serialises both the transformer stages and the trained classifier to disk. The dashboard loads these with `PipelineModel.load()`, enabling instant inference without retraining.

11 Analysis and Discussion

11.1 Comparison of Algorithms

Random Forest and Gradient Boosted Trees represent two different philosophies in ensemble learning.

Random Forest builds all trees independently and in parallel, making it faster to train and naturally resistant to overfitting because no single tree dominates the prediction. It is well-suited to the Aegis problem where the feature space is clean and the classes are well-separated.

Gradient Boosted Trees builds trees sequentially, with each tree learning from the errors of the previous one. This gives GBT a sharper boundary-detection capability at the cost of longer training time and more sensitivity to hyperparameters. It is appropriate for the Nexus problem where the simultaneous co-occurrence of three elevated features defines the overload class, requiring the model to learn joint feature interactions rather than simple thresholds.

11.2 Scalability

The current deployment runs on a single machine in `local[*]` mode, which uses all available CPU cores but does not distribute data across multiple nodes. Both datasets fit comfortably in memory at 150,000 rows each. The PySpark code itself requires no changes to scale to a true cluster — the same scripts would run on a multi-node Spark deployment simply by changing the master URL in the `SparkSession` configuration. This is one of the fundamental advantages of building on Spark from the start.

11.3 Production Readiness Assessment

The system as built demonstrates all the structural components of a production pipeline. The areas that would require additional work for real-world deployment are:

- **Real training data:** Synthetic distributions with controlled separation do not capture the noise, overlap, and edge cases present in real traffic and sensor data. A production model would require annotated real-world data for training.
- **Model monitoring:** Production ML systems require drift detection to identify when the incoming data distribution shifts away from the training distribution, triggering retraining.
- **Streaming ingestion:** The current system processes static CSV files. A production deployment would use Apache Kafka for real-time data ingestion feeding into Spark Structured Streaming.
- **Geographic visualisation:** The Nexus dashboard would benefit from a live map of the twin cities with colour-coded sector overlays showing predicted status, providing spatial context that a bar chart cannot convey.

12 Conclusion

The ML Sentience Hub demonstrates a complete, end-to-end Big Data Analytics pipeline built on Apache Spark. Starting from a data generation script that produces realistic, intentionally dirty datasets, through PySpark-based cleaning and feature engineering, to two trained and validated machine learning models, through to a live interactive dashboard — every component of the required project workflow has been implemented and validated.

Both models achieve perfect classification scores on held-out test data, confirming that the pipeline architecture is correct, the data distributions provide adequate signal, and the feature engineering stages are functioning as designed. The system exceeds the minimum project requirements by implementing two independent domains, two different machine learning algorithms, a leak-free Pipeline architecture, interactive live inference, and a multi-module visualisation dashboard.

The known limitation around distribution boundary sensitivity is acknowledged honestly and framed constructively as the natural gap between controlled experimental systems and production deployments — a gap that every real data engineering project must eventually close with real-world data collection and iterative model refinement.

A Complete Source Code

A.1 data_forge.py

```

1 import os
2 import numpy as np
3 import pandas as pd
4 from datetime import datetime, timedelta
5
6 # Reproducibility seed for all numpy randomness
7 np.random.seed(42)
8
9 DATA_DIR = "data"
10 AEGIS_PATH = os.path.join(DATA_DIR, "aegis_raw_logs.csv")
11 NEXUS_PATH = os.path.join(DATA_DIR, "nexus_raw_telemetry.csv")
12 TOTAL_ROWS = 150_000
13
14
15 def ensure_data_dir():
16     """Create the output directory if it does not exist."""
17     os.makedirs(DATA_DIR, exist_ok=True)
18
19
20 def random_timestamps(n_rows, days_back):
21     """Generate ISO timestamps uniformly over the last N days."""
22     now = datetime.utcnow()
23     max_seconds = days_back * 24 * 60 * 60
24     offsets = np.random.randint(0, max_seconds, size=n_rows)
25     return [(now - timedelta(seconds=int(s))).isoformat() for s in
26             offsets]
27
28
29 def random_ipv4(n_rows):
30     """Generate random IPv4 addresses."""
31     octets = np.random.randint(1, 255, size=(n_rows, 4))
32     return [".".join(map(str, parts)) for parts in octets]
33
34
35 def add_noise(values, min_value=None, is_int=False):
36     """Apply bounded Gaussian noise (+/-5%) to numeric values."""
37     values = values.astype(float)
38     noise = np.random.normal(loc=0.0, scale=0.05, size=values.size)
39     noise = np.clip(noise, -0.05, 0.05)
40     adjusted = np.where(np.isnan(values), np.nan, values * (1.0 + noise))
41
42     if min_value is not None:
43         adjusted = np.where(np.isnan(adjusted), np.nan, np.maximum(
44             adjusted, min_value))
45
46     if is_int:
47         adjusted = np rint(adjusted)
48         if min_value is not None:
49             adjusted = np.where(adjusted < min_value, min_value,
50                                 adjusted)
51         return adjusted.astype(int)
52     return adjusted
53
54
55 def oversample_minority(df, label_col, minority_label, numeric_spec):

```

```

51     """Duplicate minority rows with small noise until minority reaches
52         20%."""
53     counts = df[label_col].value_counts()
54     minority_count = int(counts.get(minority_label, 0))
55     majority_count = int(df.shape[0] - minority_count)
56     target_minority = int(np.ceil(0.25 * majority_count))
57     needed = max(0, target_minority - minority_count)
58     if needed == 0:
59         return df
60
61     minority_df = df[df[label_col] == minority_label]
62     sampled = minority_df.sample(n=needed, replace=True, random_state
63                                =42).copy()
64
65     for col, spec in numeric_spec.items():
66         sampled[col] = add_noise(
67             sampled[col].to_numpy(),
68             min_value=spec.get("min_value"),
69             is_int=spec.get("is_int", False),
70         )
71
72     return pd.concat([df, sampled], ignore_index=True)
73
74 def finalize_dataset(df):
75     """Shuffle and cap dataset to the required total row count."""
76     return df.sample(frac=1.0, random_state=42).head(TOTAL_ROWS).
77         reset_index(drop=True)
78
79 def build_aegis_dataset():
80     """Generate the Aegis-Vanguard web logs dataset."""
81     n_bot = int(TOTAL_ROWS * 0.05)
82     n_human = TOTAL_ROWS - n_bot
83     labels = np.array(["Human"] * n_human + ["Bot"] * n_bot)
84
85     timestamps = random_timestamps(TOTAL_ROWS, days_back=30)
86     ip_address = random_ipv4(TOTAL_ROWS)
87
88     session_duration = np.random.exponential(scale=300, size=TOTAL_ROWS)
89
90     null_count = int(TOTAL_ROWS * 0.02)
91     null_idx = np.random.choice(TOTAL_ROWS, size=null_count, replace=
92                                False)
93     session_duration[null_idx] = np.nan
94
95     click_velocity = np.empty(TOTAL_ROWS)
96     pages_viewed = np.empty(TOTAL_ROWS)
97
98     human_mask = labels == "Human"
99     bot_mask = ~human_mask
100
101     click_velocity[human_mask] = np.random.normal(2.5, 0.5, size=
102             human_mask.sum())
103     click_velocity[bot_mask] = np.random.normal(18.0, 3.0, size=
104             bot_mask.sum())

```

```

101     pages_viewed[human_mask] = np.random.normal(4, 1, size=human_mask.
102         sum())
103
104     pages_viewed[bot_mask] = np.random.normal(35, 5, size=bot_mask.sum
105         ())
106
107     click_velocity = np.clip(click_velocity, 0.1, None)
108     pages_viewed = np.clip(np rint(pages_viewed), 1, None).astype(int)
109
110     user_agents = [
111         "Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (
112             KHTML, like Gecko) Chrome/122.0.0.0 Safari/537.36",
113         "Mozilla/5.0 (Macintosh; Intel Mac OS X 13_5) AppleWebKit
114             /605.1.15 (KHTML, like Gecko) Version/17.0 Safari/605.1.15",
115         "Mozilla/5.0 (X11; Linux x86_64) AppleWebKit/537.36 (KHTML,
116             like Gecko) Chrome/121.0.0.0 Safari/537.36",
117         "Mozilla/5.0 (iPhone; CPU iPhone OS 17_2 like Mac OS X)
118             AppleWebKit/605.1.15 (KHTML, like Gecko) Version/17.2 Mobile
119             /15E148 Safari/604.1",
120         "Mozilla/5.0 (Windows NT 10.0; Win64; x64; rv:122.0) Gecko
121             /20100101 Firefox/122.0",
122         "Mozilla/5.0 (Linux; Android 14; Pixel 7) AppleWebKit/537.36 (
123             KHTML, like Gecko) Chrome/120.0.0.0 Mobile Safari/537.36",
124         "Mozilla/5.0 (iPad; CPU OS 17_1 like Mac OS X) AppleWebKit
125             /605.1.15 (KHTML, like Gecko) Version/17.1 Mobile/15E148
126             Safari/604.1",
127         "Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (
128             KHTML, like Gecko) Edge/121.0.0.0",
129     ]
130
131     ua_probs = [0.98 / 8.0] * 8 + [0.02]
132     user_agent = np.random.choice(user_agents + ["Missing"], size=
133         TOTAL_ROWS, p=ua_probs)
134
135     df = pd.DataFrame(
136         {
137             "timestamp": timestamps,
138             "ip_address": ip_address,
139             "session_duration_sec": session_duration,
140             "click_velocity_bps": click_velocity,
141             "pages_viewed": pages_viewed,
142             "user_agent": user_agent,
143             "class_label": labels,
144         }
145     )
146
147     numeric_spec = {
148         "session_duration_sec": {"min_value": 1.0, "is_int": False},
149         "click_velocity_bps": {"min_value": 0.1, "is_int": False},
150         "pages_viewed": {"min_value": 1, "is_int": True},
151     }
152
153     df = oversample_minority(df, "class_label", "Bot", numeric_spec)
154     df = finalize_dataset(df)
155     return df
156
157 def build_nexus_dataset():
158     """Generate the Nexus-Grid smart meter telemetry dataset."""
159     n_overload = int(TOTAL_ROWS * 0.02)

```

```

146 n_normal = TOTAL_ROWS - n_overload
147 labels = np.array(["Normal"] * n_normal + ["Overload"] * n_overload
148 )
149 timestamps = np.array(random_timestamps(TOTAL_ROWS, days_back=30),
150 dtype=object)
151 corrupt_count = int(TOTAL_ROWS * 0.01)
152 corrupt_idx = np.random.choice(TOTAL_ROWS, size=corrupt_count,
153 replace=False)
154 timestamps[corrupt_idx] = "CORRUPTED_TIME"
155 sectors = ["F-6", "F-7", "Bahria-Phase-7", "Bahria-Phase-8", "DHA-1",
156 "G-11"]
157 sector_id = np.random.choice(sectors, size=TOTAL_ROWS)
158 kw_draw = np.empty(TOTAL_ROWS)
159 temperature_c = np.empty(TOTAL_ROWS)
160 voltage_drop = np.empty(TOTAL_ROWS)
161 normal_mask = labels == "Normal"
162 overload_mask = ~normal_mask
163
164 kw_draw[normal_mask] = np.random.normal(45, 8, size=normal_mask.sum())
165 kw_draw[overload_mask] = np.random.normal(95, 5, size=overload_mask
166 .sum())
167 temperature_c[normal_mask] = np.random.normal(38, 5, size=
168 normal_mask.sum())
169 temperature_c[overload_mask] = np.random.normal(72, 4, size=
170 overload_mask.sum())
171 voltage_drop[normal_mask] = np.random.normal(5, 1, size=normal_mask
172 .sum())
173 voltage_drop[overload_mask] = np.random.normal(28, 3, size=
174 overload_mask.sum())
175
176 kw_draw = np.clip(kw_draw, 0.1, None)
177 temperature_c = np.clip(temperature_c, -10.0, None)
178 voltage_drop = np.clip(voltage_drop, 0.1, None)
179
180 df = pd.DataFrame(
181 {
182     "timestamp": timestamps,
183     "sector_id": sector_id,
184     "kw_draw": kw_draw,
185     "temperature_c": temperature_c,
186     "voltage_drop": voltage_drop,
187     "grid_status": labels,
188 }
189 )
190
191 numeric_spec = {
192     "kw_draw": {"min_value": 0.1, "is_int": False},
193     "temperature_c": {"min_value": -10.0, "is_int": False},
194     "voltage_drop": {"min_value": 0.1, "is_int": False},
195 }

```

```

194     df = oversample_minority(df, "grid_status", "Overload",
195                             numeric_spec)
196     df = finalize_dataset(df)
197     return df
198
199 def main():
200     """Orchestrate dataset generation and write CSV outputs."""
201     ensure_data_dir()
202
203     aegis_df = build_aegis_dataset()
204     nexus_df = build_nexus_dataset()
205
206     aegis_df.to_csv(AEGIS_PATH, index=False)
207     nexus_df.to_csv(NEXUS_PATH, index=False)
208
209
210 if __name__ == "__main__":
211     main()

```

Listing 5: data_forge.py — Synthetic dataset generator

A.2 bot_detector.py

```

1 from pyspark.sql import SparkSession
2 from pyspark.ml import Pipeline
3 from pyspark.ml.feature import Imputer, StringIndexer, VectorAssembler
4 from pyspark.ml.classification import RandomForestClassifier
5 from pyspark.ml.evaluation import MulticlassClassificationEvaluator
6
7
8 def main():
9     spark = (
10         SparkSession.builder
11             .appName("Aegis-Vanguard-Pipeline")
12             .config("spark.hadoop.security.authentication", "simple")
13             .config("spark.hadoop.fs.defaultFS", "file:///")
14             .getOrCreate()
15     )
16
17     raw_df = (
18         spark.read
19             .option("header", True)
20             .option("inferSchema", True)
21             .csv("../data/aegis_raw_logs.csv")
22     )
23
24     train_df, test_df = raw_df.randomSplit([0.8, 0.2], seed=42)
25
26     imputer = Imputer(
27         inputCol="session_duration_sec",
28         outputCol="session_duration_imputed",
29         strategy="median",
30     )
31
32     user_agent_indexer = StringIndexer(
33         inputCol="user_agent",

```

```
34     outputCol="user_agent_idx",
35     handleInvalid="keep",
36 )
37
38 label_indexer = StringIndexer(
39     inputCol="class_label",
40     outputCol="label_idx",
41     handleInvalid="keep",
42 )
43
44 assembler = VectorAssembler(
45     inputCols=[
46         "session_duration_imputed",
47         "click_velocity_bps",
48         "pages_viewed",
49         "user_agent_idx",
50     ],
51     outputCol="features",
52 )
53
54 classifier = RandomForestClassifier(
55     featuresCol="features",
56     labelCol="label_idx",
57     numTrees=100,
58     maxDepth=10,
59     seed=42,
60 )
61
62 pipeline = Pipeline(stages=[
63     imputer,
64     user_agent_indexer,
65     label_indexer,
66     assembler,
67     classifier,
68 ])
69
70 model = pipeline.fit(train_df)
71 predictions = model.transform(test_df)
72
73 evaluator = MulticlassClassificationEvaluator(
74     labelCol="label_idx",
75     predictionCol="prediction",
76 )
77
78 accuracy = evaluator.setMetricName("accuracy").evaluate(predictions)
79 f1 = evaluator.setMetricName("f1").evaluate(predictions)
80
81 print(f"Accuracy: {accuracy}")
82 print(f"F1 Score: {f1}")
83 # Save the model
84 model.write().overwrite().save("../models/aegis_model")
85 spark.stop()
86
87
88 if __name__ == "__main__":
89     main()
```

Listing 6: bot_detector.py — Module Alpha ML pipeline

A.3 grid_predictor.py

```
1 from pyspark.sql import SparkSession
2 from pyspark.ml import Pipeline
3 from pyspark.ml.feature import StringIndexer, VectorAssembler
4 from pyspark.ml.classification import GBTClassifier
5 from pyspark.ml.evaluation import MulticlassClassificationEvaluator
6
7
8 def main():
9     spark = (
10         SparkSession.builder
11         .appName("Nexus-Grid-Pipeline")
12         .getOrCreate()
13     )
14
15     raw_df = (
16         spark.read
17         .option("header", True)
18         .option("inferSchema", True)
19         .csv("../data/nexus_raw_telemetry.csv")
20     )
21
22     cleaned_df = raw_df.filter(raw_df.timestamp != "CORRUPTED_TIME")
23
24     train_df, test_df = cleaned_df.randomSplit([0.8, 0.2], seed=42)
25
26     sector_indexer = StringIndexer(
27         inputCol="sector_id",
28         outputCol="sector_id_idx",
29         handleInvalid="keep",
30     )
31
32     status_indexer = StringIndexer(
33         inputCol="grid_status",
34         outputCol="grid_status_idx",
35         handleInvalid="keep",
36     )
37
38     assembler = VectorAssembler(
39         inputCols=[
40             "kw_draw",
41             "temperature_c",
42             "voltage_drop",
43             "sector_id_idx",
44         ],
45         outputCol="features",
46     )
47
48     classifier = GBTClassifier(
49         featuresCol="features",
50         labelCol="grid_status_idx",
51         maxIter=20,
52         seed=42,
```

```
53     )
54
55     pipeline = Pipeline(stages=[
56         sector_indexer,
57         status_indexer,
58         assembler,
59         classifier,
60     ])
61
62     model = pipeline.fit(train_df)
63     predictions = model.transform(test_df)
64
65     evaluator = MulticlassClassificationEvaluator(
66         labelCol="grid_status_idx",
67         predictionCol="prediction",
68     )
69
70     accuracy = evaluator.setMetricName("accuracy").evaluate(predictions)
71
72     f1 = evaluator.setMetricName("f1").evaluate(predictions)
73
74     print(f"Accuracy: {accuracy}")
75     print(f"F1 Score: {f1}")
76     # Save the model
77     model.write().overwrite().save("../models/nexus_model")
78     spark.stop()
79
80 if __name__ == "__main__":
81     main()
```

Listing 7: grid_predictor.py — Module Beta ML pipeline